Epic 3: Data Pre-processing

Story 5: Feature Scaling

Feature scaling is an important preprocessing step that standardizes the numerical input features to a common scale before training machine learning models. Without feature scaling, variables with larger numerical values may dominate those with smaller values, leading to biased learning and reduced model performance.

StandardScaler

In this project, the StandardScaler class from the Scikit-learn library is used to standardize the independent variables. It transforms each feature by subtracting the mean and dividing by the standard deviation.

Formula:

$$z = \frac{x - \mu}{\sigma}$$

Where:

- **x = Original feature value**
- **μ = Mean of the feature**
- **σ = Standard deviation of the feature**

After scaling, each feature has a mean close to 0 and a standard deviation close to 1, allowing all input variables to contribute equally during model training.

Application

Feature scaling is applied only to the independent variables (X). The target variable (y) is not scaled because it represents the class label (Flood or No Flood) rather than a numerical feature.

The scaler is fitted using the training dataset and then applied to both the training and testing datasets to maintain consistency.

Example Implementation

```
from sklearn.preprocessing import StandardScaler


sc = StandardScaler()


X_train = sc.fit_transform(X_train)
```

X_test = sc.transform(X_test)

Saving the Scaler

The fitted StandardScaler object is saved using Joblib or Pickle. During deployment, the same scaler is loaded and applied to new user input before prediction. This ensures that real-time input data is processed in exactly the same way as the training data.

Benefits

- **Standardizes numerical features to a common scale.**
- **Improves the performance and stability of machine learning models.**
- **Prevents features with larger values from dominating the learning process.**
- **Essential for algorithms such as K-Nearest Neighbours (KNN), Logistic Regression, and Support Vector Machines (SVM).**
- **Ensures consistent preprocessing during both training and deployment.**

Outcome

- **Applied StandardScaler to standardize the input features.**
- **Prepared the training and testing datasets for machine learning.**
- **Saved the fitted scaler for real-time prediction in the Flask application.**
- **Improved model consistency and prediction accuracy.**

Figure 11: Applying StandardScaler to the training and testing datasets before model training.

```
#import Standardscaler
from sklearn.preprocessing import StandardScaler
#create object to Standardscaler class
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.fit_transform(x_test)
```